Justificativa do Backend Stateless para Escalabilidade

Projeto: OrbiVet
Fase 4: System Design e Infraestrutura
Subtarefa: Justificar como o backend stateless ajudará a absorver a proporção de leitura/escrita.

1. Arquitetura Stateless

O OrbiVet utilizará um backend REST stateless para facilitar a escalabilidade do sistema e permitir que diferentes servidores processem as requisições dos usuários de forma independente.

Em uma arquitetura stateless, o servidor não armazena informações de sessão do usuário em sua própria memória para atender às próximas requisições. Cada solicitação contém os dados necessários para ser processada, enquanto as informações que precisam ser mantidas, como cadastros de animais, usuários e históricos de saúde, ficam armazenadas no banco de dados.

Na prática, isso significa que um tutor conseguirá consultar os dados do seu pet em uma instância do backend e, na próxima requisição, ser atendido por outra, sem que isso prejudique o funcionamento da aplicação.

A autenticação seguirá esse modelo, utilizando tokens JWT enviados nas requisições e validados pelo backend, sem depender de sessões armazenadas localmente.

2. Como o Stateless Ajudará na Escalabilidade

A principal vantagem dessa arquitetura é a escalabilidade horizontal, ou seja, a possibilidade de adicionar novas instâncias da aplicação conforme a quantidade de acessos aumenta.

Para isso, o OrbiVet vai usar um Load Balancer, que vai ser responsável por distribuir as requisições entre os servidores disponíveis.

Por exemplo, quando muitos usuários acessarem o OrbiVet ao mesmo tempo para consultar animais perdidos, visualizar históricos de vacinação ou realizar agendamentos, essas requisições não ficarão Armazenadas em um único servidor.

Como as instâncias serão independentes, a carga será distribuída entre elas. Com base no aumento da demanda, novas instâncias serão adicionadas para aumentar a capacidade de processamento.

Outro benefício será a disponibilidade. Se uma instância apresentar problemas, o balanceador encaminhará novas requisições para outras instâncias ativas, evitando que a falha de um único servidor interrompa todo o atendimento.

3. Relação com a Proporção de Leitura e Escrita

No OrbiVet, as leituras acontecem quando o usuário consulta o perfil de um animal, vê as vacinas registradas, procura clínicas ou busca um pet perdido. Já as escritas acontecem quando alguém cadastra um animal, atualiza seus dados, registra uma vacina ou marca uma consulta.

A previsão é que, a cada 3 operações realizadas no sistema, aproximadamente 2 sejam leituras e 1 seja uma escrita. Por isso, o OrbiVet precisa conseguir atender várias requisições ao mesmo tempo sem comprometer seu funcionamento.

Para ajudar nisso, o backend stateless permitirá distribuir as requisições entre diferentes instâncias da API. Assim, em vez de depender de um único servidor para processar tudo, o sistema poderá dividir esse trabalho e adicionar mais instâncias conforme a quantidade de acessos aumentar.

Na prática, quando alguém consultar as informações de um animal, a API vai buscar os dados no banco e retornar o resultado. Quando houver um cadastro ou alguma atualização, ela vai validar as informações e enviá-las ao banco para serem salvas.

Mas só aumentar o número de instâncias não resolve tudo, já que o banco de dados também pode ficar sobrecarregado com tantas operações. Por isso, o projeto também vai utilizar cache, réplicas de leitura e melhorias nas consultas ao banco.

O cache será usado para evitar que o sistema precise buscar as mesmas informações no banco toda vez que alguém fizer uma consulta. Já as réplicas de leitura ajudarão a dividir o volume de consultas, deixando o banco principal com menos trabalho. Os cadastros e as atualizações continuarão sendo feitos no banco principal, para manter os dados corretos.

Com isso, a ideia é que o OrbiVet consiga atender mais usuários sem depender apenas de um aumento na capacidade dos servidores. Cada solução vai ajudar de uma forma diferente, tanto na distribuição das requisições quanto na redução da sobrecarga do banco de dados.

4. Limitações e Cuidados

Mesmo com várias instâncias do backend funcionando, todas ainda vão depender dos mesmos serviços, principalmente do banco de dados. Por isso, é importante garantir que ele consiga lidar com a quantidade de consultas e gravações feitas pelo sistema.

Também precisamos pensar nos momentos em que muitos usuários acessam o sistema ao mesmo tempo, como quando um animal perdido é divulgado e várias pessoas entram na plataforma para buscar informações.

Por esse motivo, não tem que apenas distribuir as requisições entre as instâncias. Também vai ser necessário acompanhar o desempenho do banco de dados, o número de conexões, o tempo de resposta e o consumo de recursos da aplicação.

Outro ponto importante é evitar que informações necessárias para o funcionamento do sistema fiquem armazenadas apenas na memória de uma instância. Porque, se ela for reiniciada ou substituída, esses dados podem ser perdidos.

Para garantir que tudo funcione, vai ser realizado vários testes de carga, simulando diferentes quantidades de usuários acessando o sistema ao mesmo tempo. Com esses testes, será possível identificar futuros problemas de desempenho e descobrir se adicionar mais instâncias realmente vai ajudar a melhorar o funcionamento do OrbiVet.

5. Conclusão

A escolha de um backend REST stateless para o OrbiVet foi pensada para facilitar o crescimento do sistema e permitir que as requisições sejam distribuídas entre diferentes instâncias da aplicação.

Com isso, o sistema poderá lidar melhor com uma quantidade de 1 milhão de escritas para cada 2 milhões de leituras, permitindo aumentar a capacidade do backend conforme a demanda crescer. Como ele não depende de sessões armazenadas em um servidor específico, fica mais fácil distribuir as requisições e adicionar novas instâncias quando necessário.

Assim, o uso de um backend stateless ajuda a preparar o OrbiVet para crescer, distribuindo melhor as requisições e reduzindo a dependência de servidores específicos. Dessa forma, o sistema terá uma estrutura mais preparada para lidar com o aumento de usuários e operações ao longo do tempo.